PROJECT AND TEAM INFORMATION

Project Title

Smart Hostel Management & Complaint Resolution System

Team Name: Hostel -Nexus **Team #**	**DSCPP-III-2026-T269**
Team member 1 (Team Lead)	Raj, Harshit - 2510011339 *rajharshit1807@gmail.com* 
Team member 2	Negi, Diviya - 2510010573 *diviyanegi51@gmail.com* 
Team member 3	Verma, Aditya 2510380258 *av0844769@gmail.com* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Traditional hostel administration relies heavily on scattered physical logbooks and disjointed spreadsheets. This setup regularly leads to misplaced leave applications, unattended student grievances, and slow room allocation. To solve this, we engineered the Smart Hostel Management System—a unified console application built in C and C++ designed to centralize core administrative workflows into one coherent pipeline.

The architecture breaks down into five interconnected modules: System Core (Main), Mess & Dining, Complaint Tracking, Student Leave, and Room Inventory. Students can file grievances directly, review room allocations, and log exit requests without administrative delays. In the complaint engine, an automated triage system evaluates grievances based on urgency so that critical room or utility breakdowns get resolved before routine inquiries.

Technically, the backend utilizes core computer science fundamentals—specifically standard file I/O for persistent disk storage and specialized data structures (linked lists, FIFO queues, max-heaps, and hash maps) to ensure efficient record lookup and memory handling. Our primary milestone was turning theoretical data structure paradigms into a functional, terminal-driven utility that eliminates administrative overhead and automates daily hostel record-keeping.

Updated Project Approach and Architecture

Smart Hostel Management System is structured into five modular components using C and C++:

- **Main Module (main.cpp):** Connects all modules and provides a menu to access different hostel management features.
- **Food and Mess Module:** Manages food and mess-related information and activities using functions and file handling.
- **Complaint Module (complaint.c):** Uses a FIFO queue for normal complaints, a priority queue implemented using a binary max-heap for urgent complaints, and a linked list to maintain resolved complaint history.
- **Hostel Leave Module:** Manages student leave requests and maintains leave-related records.
- **Room Availability Module:** Manages room details and helps check room availability.
- **Data Storage and Integration:** Uses file handling to store records and header files to connect the modules. Standard libraries such as stdio.h, stdlib.h, string.h, and iostream support input/output, memory management, and string operations.

The application relies on dedicated standalone translation units coordinated by a centralized menu driver. This decoupled architecture isolates module errors, streamlines record updates, and keeps daily administrative tasks responsive and well-structured.

Tasks Completed

Task Completed	Team Member
1.Main module development and menu integration (main.cpp)	Harshit
2.Food and Mess module development	Harshit
3.Complaint module development using queues, priority queues, linked lists, and file handling	Diviya
4.Hostel Leave module development	Aditya
5.Room Availability module development	Aditya
6.Implementation of data structures and file handling for managing hostel records	All team members
7.Basic integration of the five modules using functions and header files	All team members

Challenges/Roadblocks

1.Integration of C and C++ Modules: Connecting different modules and making them work together was difficult. We addressed this by using proper header files, function declarations, and compilation commands.

2. File Handling and Data Storage: Saving and retrieving records across multiple modules led to formatting bugs and dangling file pointers. We resolved this by building structured serialization helpers and enforcing EOF validation before every record read.

3. Complaint Priority Management: Coordinating emergency escalations alongside standard FIFO queues demanded strict pointer and heap maintenance. We implemented a binary max-heap with explicit sift-up and sift-down routines to retrieve top-urgency complaints in O(1) time.

4. Testing and Debugging: Cross-compiling modular headers caused unexpected runtime exceptions and boundary errors. We eliminated these by developing isolated test-driver scripts for each individual module before linking them into the full executable.

Task Pending	Team Member
1. Final testing and error fixing in the Complaint Module	Diviya
2. Final testing and error fixing in the Food and Mess Module	Harshit
3. Checking file handling and record storage	Diviya and Harshit
4. Integration and testing of all five modules	All team members
5. Final system testing and debugging	All team members

Project Outcome/Deliverables

The finished system delivers an integrated command-line suite that coordinates all everyday hostel operations through a clean, menu-driven CLI interface. Rather than juggling separate logbooks, hostel wardens can manage dining schedules, student leaves, occupancy records, and student grievances from a single binary.

Core deliverables incorporate robust data management: FIFO queues process normal student requests in order of submission, a binary max-heap surfaces critical infrastructure complaints instantly, and linked list nodes maintain historical query logs. Hash-based key indexing ensures swift searches when querying room occupancies.

The final deliverable is an efficient, console-driven application that automates administrative tasks and keeps records secure. The project illustrates how classic data structures can solve practical concurrency, scheduling, and record-retrieval challenges in a real-world software system.

Progress Overview

Phase-II development has reached its functional implementation targets. All five specialized components—Main, Mess, Complaint Handling, Leave Management, and Room Capacity—are fully written and verified as individual modules. We bridged the underlying C algorithmic logic with C++ interface structures, ensuring reliable cross-module calls. Key memory constructs, including heap structures, queues, dynamic chains, and lookup arrays, are integrated alongside basic disk persistence.

Our current engineering sprint is centered around integration testing: catching link-time mismatches, eliminating loose memory leaks, and harmonizing console transitions. Because our persistent files interact across multiple modules, verifying that simultaneous reads and writes do not corrupt shared state has required dedicated attention. Development remains on track, with remaining efforts focused on buffer sanitization, comprehensive error handling, and complete binary hardening.

Codebase Information

Repository: https://github.com/rajharshit1807-stack/3rd-Sem--PBL.git

Source code collaboration is organized through GitHub using feature-branch workflows to isolate individual module updates before staging pull requests into main. The solution unifies five core functional units—Main Controller, Dining & Mess, Priority Complaints, Student Exit/Leave, and Room Occupancy Tracking. Leveraging targeted data structures (binary max-heaps, dynamic linked lists, FIFO queues, and hash-indexed tables), the codebase uses C++ object-oriented principles (data encapsulation, struct modularity, separate header declarations) paired with standard disk I/O routines for persistent state management across runs.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Main menu testing	In Progress	Checking access to different modules.
Complaint module testing	Pass	Complaint registration, priority handling, and resolution features tested.
Food and Mess module testing	Pass	Food and mess-related functions tested successfully.
File handling testing	In Progress	Checking whether records are saved and retrieved correctly.
Module integration testing	In Progress	Checking connections between all five modules.
Final system testing	Pending	Final testing will be completed after resolving remaining issues.

Deliverables Progress

1. Main Module – In Progress
2. Food and Mess Module – Completed
3. Complaint Module – In Progress (testing and error fixing)
4. Hostel Leave Module – Completed
5. Room Availability Module – Completed
6. C and C++ Module Integration – In Progress
7. File Handling and Data Storage – In Progress
8. Final Testing and Debugging – Pending